به نام خدا

طراحی و شبیه‌سازی سیستم تشخیص حرکت و اعلام هشدار با سنسور PIR مبتنی برARM Cortex-M3

پروژه پایانی درس ریزپردازنده و زبان اسمبلی

نام درس: ریزپردازنده و زبان اسمبلی

عنوان پروژه: طراحی و شبیه‌سازی سیستم تشخیص حرکت و اعلام هشدار با سنسورPIR

پردازنده انتخابیARM Cortex-M3 / STM32F103C8T6

محیط پیاده‌سازی و شبیه‌سازیKeil uVision

زبان‌های استفاده‌شده C وARM Assembly

نام استاد :دکتر مقدسی

اعضای گروه:

عطیه جعفرزاده (شماره دانشجویی: 402103206) - سپیده پاشایان (شماره دانشجویی: 402103242)

نام دانشگاه :دانشگاه صنعتی قم

نیمسال تحصیلی :نیمسال دوم

1) تحلیل سیستم و طراحی سطح بالا

1-1 . تعریف مسئله

در بسیاری از محیط‌ها، نیاز است ورود یا حرکت افراد به صورت خودکار تشخیص داده شود و سیستم بدون دخالت انسان، نسبت به آن واکنش نشان دهد. برای مثال در یک اتاق، راهرو، انبار، آزمایشگاه یا محیط امنیتی، تشخیص حرکت می‌تواند باعث فعال شدن هشدار، روشن شدن چراغ یا ثبت وضعیت شود. در این پروژه هدف طراحی و شبیه‌سازی یک سیستم نهفته ساده برای تشخیص حرکت و اعلام هشدار است.

سیستم طراحی‌شده بر اساس یک سنسور PIR کار می‌کند. سنسور PIR یک سنسور تشخیص حرکت است که معمولاً برای تشخیص حرکت بدن انسان یا اجسام دارای تابش حرارتی استفاده می‌شود. خروجی این سنسور به صورت دیجیتال در نظر گرفته شده است؛ یعنی در حالت عادی مقدار صفر و در زمان تشخیص حرکت مقدار یک تولید می‌کند. این سیگنال به میکروکنترلر ارسال می‌شود و میکروکنترلر بر اساس وضعیت ورودی، خروجی‌های سیستم را کنترل می‌کند.

در این پروژه از یک میکروکنترلر مبتنی بر معماری ARM Cortex-M3 استفاده شده است. برنامه اصلی سیستم در زبان C نوشته شده و بخش کلیدی پردازش وضعیت سنسور در زبان اسمبلی ARM پیاده‌سازی شده است. به این ترتیب پروژه علاوه بر جنبه کاربردی، ارتباط مستقیم با مباحث درس ریزپردازنده و زبان اسمبلی دارد. مفاهیمی مانند کار با رجیسترها، پورت‌های ورودی/خروجی، دستورات شرطی، فراخوانی زیرروال، استفاده ازStack ، تایمر SysTickو ارتباط بین زبان C و اسمبلی در این پروژه به کار گرفته شده‌اند.

مسئله اصلی پروژه این است که سیستم بتواند وضعیت سنسور PIR را به صورت پیوسته بررسی کند. اگر حرکت تشخیص داده شد، سیستم باید وارد حالت هشدار شود و خروجی‌هایی مانند چراغ قرمز و بازر را فعال کند. اگر حرکت تشخیص داده نشد، سیستم باید در حالت عادی باقی بماند و چراغ سبز فعال باشد. همچنین برای جلوگیری از واکنش‌های لحظه‌ای و ناپایدار، یک پردازش ساده روی وضعیت سنسور انجام می‌شود تا نتیجه قابل اعتمادتر باشد.

1-2.شرح کاربرد سیستم

سیستم تشخیص حرکت طراحی‌شده در این پروژه می‌تواند به عنوان یک نمونه ساده از سیستم‌های امنیتی و کنترلی مورد استفاده قرار گیرد. نمونه‌های واقعی این سیستم در خانه‌های هوشمند، سیستم‌های هشدار، راهروهای ساختمان، پارکینگ‌ها، انبارها و محیط‌های صنعتی دیده می‌شود. در چنین سیستم‌هایی، لازم نیست کاربر به صورت دستی وضعیت محیط را بررسی کند؛ بلکه سیستم به کمک سنسور و پردازنده، تغییرات محیط را تشخیص داده و خروجی مناسب تولید می‌کند.

در حالت عادی، سیستم هیچ حرکتی را تشخیص نداده و چراغ سبز روشن است. روشن بودن چراغ سبز نشان می‌دهد که سیستم فعال است اما وضعیت خطر یا هشدار وجود ندارد. زمانی که سنسور PIR حرکت را تشخیص دهد، وضعیت ورودی تغییر می‌کند. این مقدار توسط میکروکنترلر خوانده شده و سپس به زیرروال اسمبلی ارسال می‌شود. زیرروال اسمبلی مقدار ورودی را بررسی کرده و نتیجه را به برنامه اصلی برمی‌گرداند. اگر نتیجه نشان‌دهنده تشخیص حرکت باشد، برنامه اصلی خروجی هشدار را فعال می‌کند. در این حالت چراغ قرمز روشن شده و بازر نیز فعال می‌شود.

این سیستم از نظر آموزشی اهمیت زیادی دارد، زیرا یک نمونه واقعی از ترکیب سخت‌افزار و نرم‌افزار در سیستم‌های نهفته است. در سطح سخت‌افزار، پایه‌های ورودی و خروجی، رجیسترهای GPIO و تایمر سیستم مورد استفاده قرار می‌گیرند. در سطح نرم‌افزار، برنامه C وظیفه مدیریت کلی سیستم را بر عهده دارد و روتین اسمبلی بخش تصمیم‌گیری سریع و مستقیم را انجام می‌دهد. بنابراین دانشجو می‌تواند ارتباط بین مفاهیم تئوری درس و یک کاربرد عملی را مشاهده کند.

1-3 . ورودی‌ها، پردازش و خروجی‌های سیستم

هر سیستم نهفته معمولاً شامل سه بخش اصلی است: ورودی، پردازش و خروجی. در این پروژه ورودی سیستم، سنسور PIRاست. این سنسور به پایه PA0 متصل شده و وضعیت حرکت را به صورت سیگنال دیجیتال به میکروکنترلر ارسال می‌کند.
بخش پردازش سیستم توسط میکروکنترلر ARM Cortex-M3 انجام می‌شود. میکروکنترلر ابتدا پورت‌های ورودی و خروجی را مقداردهی اولیه می‌کند. سپس در حلقه اصلی برنامه، وضعیت سنسور را به صورت مداوم بررسی می‌کند. مقدار خوانده‌شده از سنسور به تابع اسمبلی ارسال می‌شود. این تابع وظیفه دارد مقدار ورودی را بررسی کرده و مشخص کند که آیا حرکت تشخیص داده شده است یا خیر. نتیجه پردازش دوباره به برنامه C بازگردانده می‌شود.

خروجی‌های سیستم شامل LED سبز، LEDقرمز و Buzzer هستند LED .سبز برای نمایش حالت عادی سیستم استفاده می‌شود LED .قرمز برای نمایش حالت هشدار به کار می‌رود Buzzer .نیز در زمان تشخیص حرکت فعال می‌شود تا علاوه بر نمایش بصری، اعلام هشدار صوتی نیز انجام شود.

در این پروژه پایه‌های خروجی به این شکل در نظر گرفته شده‌اند:
پایه PA1 برای LED سبز
پایه PA2 برایBuzzer
پایه PC13 برای LED قرمز
پایه PA0 برای ورودی سنسورPIR
بنابراین مسیر کلی داده در سیستم به این صورت است: ابتدا وضعیت محیط توسط سنسور PIR تشخیص داده می‌شود. سپس مقدار سنسور وارد میکروکنترلر می‌شود. میکروکنترلر مقدار ورودی را پردازش می‌کند و در نهایت بر اساس نتیجه، خروجی مناسب را رویLED ها و Buzzer اعمال می‌کند.

1-4 . نیازمندی‌های عملکردی سیستم

نیازمندی‌های عملکردی مشخص می‌کنند که سیستم باید چه کارهایی انجام دهد. مهم‌ترین نیازمندی‌های عملکردی پروژه عبارت‌اند از:

سیستم باید بتواند پایه ورودی متصل به سنسور PIR را بخواند. مقدار این ورودی مشخص می‌کند که حرکت در محیط تشخیص داده شده است یا خیر. اگر خروجی سنسور مقدار صفر داشته باشد، سیستم باید حالت عادی را نمایش دهد. اگر خروجی سنسور مقدار یک داشته باشد، سیستم باید وارد حالت هشدار شود.

سیستم باید بتواند خروجی‌های خود را کنترل کند. در حالت عادی، LEDسبز باید روشن باشد و LED قرمز و Buzzerخاموش باشند. در حالت هشدار، LEDسبز خاموش شده و LED قرمز و Buzzer فعال می‌شوند. این تغییر وضعیت باید به صورت خودکار و بر اساس مقدار سنسور انجام شود.

یکی دیگر از نیازمندی‌های مهم سیستم، استفاده از یک بخش اسمبلی در پردازش داده است. از آنجا که هدف پروژه فقط نوشتن برنامه C نیست، بخشی از تصمیم‌گیری سیستم در زبان اسمبلی ARM نوشته شده است. این بخش مقدار سنسور را دریافت کرده، آن را بررسی می‌کند و نتیجه را به برنامه C برمی‌گرداند.

همچنین سیستم باید از تایمر SysTick برای مدیریت زمان استفاده کند. در این پروژه از SysTick برای ایجاد

زمان‌بندی و کنترل مدت فعال بودن هشدار استفاده می‌شود. به عنوان مثال، پس از تشخیص حرکت، سیستم می‌تواند هشدار را برای چند ثانیه فعال نگه دارد و سپس دوباره وضعیت سنسور را بررسی کند.

در نهایت، سیستم باید در محیط IDE قابل Build و شبیه‌سازی باشد. فایل‌های C و Assembly باید به صورت صحیح در پروژه قرار گرفته باشند و برنامه بدون خطا کامپایل شود. این موضوع نشان می‌دهد که ارتباط بین بخش C و اسمبلی به درستی انجام شده است.

1-5 . محدودیت‌های سیستم

سیستم طراحی‌شده یک مدل ساده‌شده از یک سیستم تشخیص حرکت واقعی است. بنابراین برخی محدودیت‌ها در آن وجود دارد. اولین محدودیت این است که پروژه در محیط شبیه‌سازی پیاده‌سازی شده و ممکن است به صورت عملی روی سخت‌افزار واقعی تست نشده باشد. در نتیجه رفتار واقعی سنسورPIR ، نویز محیط، تأخیر سنسور و شرایط فیزیکی محیط به طور کامل در شبیه‌سازی دیده نمی‌شود.
محدودیت دوم این است که سنسور PIR به صورت یک ورودی دیجیتال ساده مدل شده است. در سیستم‌های واقعی، خروجی سنسور ممکن است تحت تأثیر نویز، دمای محیط یا حرکت‌های ناخواسته قرار بگیرد. برای کاهش خطا در سیستم واقعی، معمولاً از روش‌هایی مانند فیلتر نرم‌افزاری، تأخیر زمانی یا چند بار نمونه‌برداری از ورودی استفاده می‌شود.
محدودیت دیگر مربوط به خروجی‌ها است. در این پروژه خروجی‌ها به LED و Buzzer محدود شده‌اند. در یک سیستم واقعی می‌توان خروجی‌های دیگری مانند ارسال پیام، فعال کردن رله، ثبت زمان حرکت در حافظه یا ارتباط با سیستم مرکزی را نیز اضافه کرد. اما برای حفظ سادگی پروژه و تمرکز روی مفاهیم درس، خروجی‌ها به چند بخش ساده محدود شده‌اند.
همچنین در این پروژه تمرکز اصلی روی تحلیل، طراحی، مدل‌سازی C و پیاده‌سازی بخش کلیدی در اسمبلی است. بنابراین پروژه از نظر سخت‌افزاری کاملاً صنعتی نیست، بلکه یک نمونه آموزشی برای درک ارتباط بین نرم‌افزار سطح بالا و اسمبلی در یک سیستم نهفته محسوب می‌شود.

1-6 . چرخه حیات سیستم

چرخه حیات این سیستم شامل چهار مرحله تحلیل، طراحی، پیاده‌سازی و آزمون است.
در مرحله تحلیل، ابتدا هدف پروژه مشخص شد. هدف طراحی سیستمی بود که بتواند حرکت را از طریق سنسور PIR تشخیص دهد و در صورت نیاز هشدار تولید کند. سپس ورودی‌ها، خروجی‌ها، نیازمندی‌های عملکردی و محدودیت‌های سیستم تعیین شدند.
در مرحله طراحی، ساختار کلی سیستم مشخص شد. در این مرحله نمودار بلوکی سیستم طراحی شد تا ارتباط بین سنسور، میکروکنترلر و خروجی‌ها نشان داده شود. سپس گراف جریان داده یا DFD طراحی شد تا مسیر حرکت داده از محیط تا خروجی هشدار مشخص شود. همچنین فلوچارت الگوریتم اصلی طراحی شد تا روند اجرای برنامه از مقداردهی اولیه تا حلقه اصلی و تصمیم‌گیری مشخص باشد.
در مرحله پیاده‌سازی، برنامه اصلی سیستم در زبان C نوشته شد. در این برنامه، رجیسترهای GPIO مقداردهی اولیه شدند، وضعیت سنسور خوانده شد، خروجی‌ها کنترل شدند و تایمر SysTick برای مدیریت زمان استفاده شد. علاوه بر آن، یک فایل اسمبلی با نام.motion_asm نوشته شد که وظیفه پردازش کلیدی وضعیت سنسور را بر عهده دارد.
در مرحله آزمون، پروژه در محیط Keil uVision ساخته و بررسی شد. فایل‌های C و Assembly در پروژه قرار گرفتند و نتیجه Build نشان داد که برنامه بدون خطا کامپایل شده است. همچنین در محیط شبیه‌سازی، وضعیت خروجی‌ها و منطق عملکرد سیستم بررسی شد تا مشخص شود برنامه در حالت عادی و حالت هشدار رفتار درستی دارد.

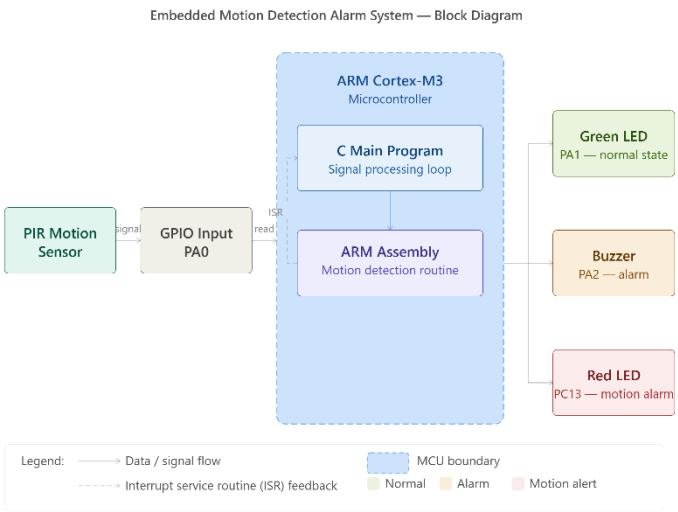
1-7 .تقسیم سیستم به ماژول‌ها

برای طراحی ساختاریافته، سیستم به چند ماژول اصلی تقسیم شده است. این ماژول‌بندی باعث می‌شود برنامه خواناتر، قابل توسعه‌تر و قابل بررسی‌تر باشد.
ماژول اول، ماژول مقداردهی اولیه سیستم است. این ماژول وظیفه دارد تنظیمات اولیه مورد نیاز برای اجرای برنامه را انجام دهد. فعال‌سازی کلاک پورت‌ها، تنظیم پایه‌های ورودی و خروجی و آماده‌سازی تایمر از وظایف این بخش است.
ماژول دوم، ماژول خواندن سنسور PIR است. این بخش وضعیت پایه PA0 را بررسی می‌کند و مقدار ورودی سنسور را دریافت می‌کند. مقدار خوانده‌شده سپس برای پردازش به بخش اسمبلی ارسال می‌شود.
ماژول سوم، ماژول پردازش اسمبلی است. این ماژول در فایل motion_asm.s قرار دارد و وظیفه آن بررسی مقدار ورودی و تعیین وضعیت حرکت است. استفاده از اسمبلی در این بخش باعث می‌شود مفاهیم رجیستر، دستورات مقایسه، پرش شرطی، Load/Storeو بازگشت از زیرروال به صورت عملی در پروژه استفاده شوند.
ماژول چهارم، ماژول کنترل خروجی‌ها است. این ماژول بر اساس نتیجه پردازش، LEDها و Buzzer را کنترل می‌کند. اگر حرکت تشخیص داده شود، خروجی‌های هشدار فعال می‌شوند و اگر حرکتی وجود نداشته باشد، سیستم در وضعیت عادی قرار می‌گیرد.
ماژول پنجم، ماژول زمان‌بندی با SysTick است. این بخش برای ایجاد زمان‌بندی میلی‌ثانیه‌ای و کنترل مدت هشدار استفاده می‌شود. وجود این ماژول باعث می‌شود برنامه بتواند رفتار زمانی قابل کنترل داشته باشد.
در نهایت، ماژول حلقه اصلی برنامه وظیفه هماهنگ کردن همه بخش‌ها را بر عهده دارد. این حلقه به صورت مداوم ورودی را بررسی می‌کند، پردازش اسمبلی را فراخوانی می‌کند و خروجی مناسب را فعال می‌سازد.

2.نمودارهای سیستم

2-1 . نمودار بلوکی سیستم

در نمودار بلوکی، اجزای اصلی سیستم و ارتباط بین آن‌ها نمایش داده می‌شود. این نمودار نشان می‌دهد که سنسور PIRبه عنوان ورودی به میکروکنترلر متصل است و میکروکنترلر پس از پردازش داده، خروجی‌هایی مانند LED سبز، LEDقرمز و Buzzer را کنترل می‌کند.



2-2 .گراف جریان دادهDFD

در گراف جریان داده، مسیر حرکت اطلاعات در سیستم نمایش داده می‌شود. در این پروژه، داده از محیط وارد سنسور PIR می‌شود. سنسور مقدار دیجیتال تولید می‌کند. این مقدار توسط میکروکنترلر خوانده می‌شود و سپس در بخش پردازش، وضعیت حرکت مشخص می‌شود. نتیجه پردازش به بخش کنترل خروجی ارسال شده و در نهایت LEDها و Buzzer فعال یا غیرفعال می‌شوند.

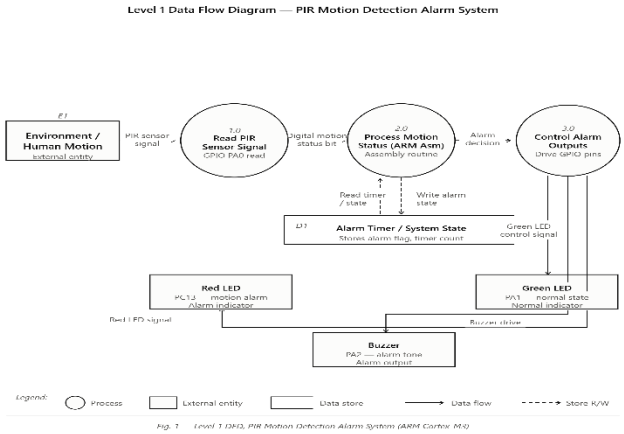


Fig. 1 Level 1 DFD, PIR Motion Detection Alarm System (ARM Cortex M4)

2-3 . فلوچارت الگوریتم اصلی

فلوچارت الگوریتم اصلی نشان می‌دهد که برنامه از چه مراحلی عبور می‌کند. ابتدا سیستم شروع می‌شود و پورت‌ها و تایمر مقداردهی اولیه می‌شوند. سپس برنامه وارد حلقه اصلی می‌شود. در این حلقه وضعیت سنسور PIR خوانده می‌شود و تابع اسمبلی برای تشخیص حرکت فراخوانی می‌گردد. اگر حرکت تشخیص داده شود، حالت هشدار فعال می‌شود. اگر حرکتی تشخیص داده نشود، سیستم در حالت عادی قرار می‌گیرد. سپس برنامه دوباره به ابتدای حلقه برمی‌گردد.

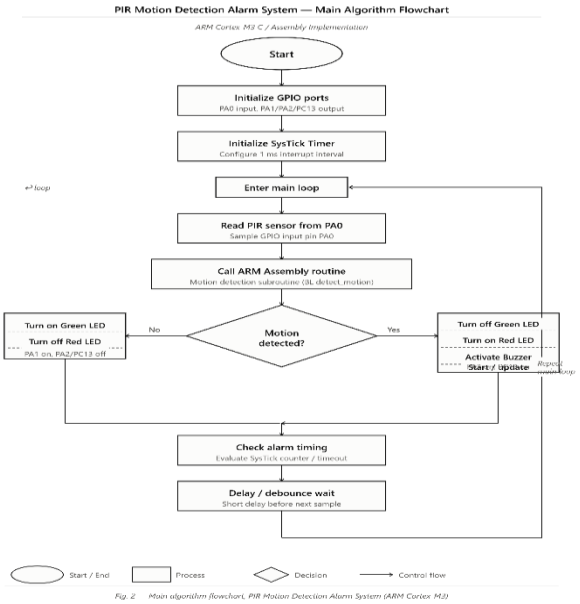


Fig. 2 Main algorithm flowchart, PIR Motion Detection Alarm System (ARM Cortex M3)

2-4 . نمودار ماژول‌بندی نرم‌افزار

در این نمودار، بخش‌های نرم‌افزاری پروژه نمایش داده می‌شود. برنامه اصلی در زبان C نوشته شده و چند بخش مختلف را مدیریت می‌کند. یک بخش وظیفه مقداردهی اولیه GPIO را دارد. بخش دیگر وضعیت سنسور را می‌خواند. بخش اسمبلی وظیفه پردازش وضعیت سنسور را دارد. بخش کنترل خروجی، LEDها و Buzzer را مدیریت می‌کند و بخش SysTick زمان‌بندی سیستم را انجام می‌دهد.

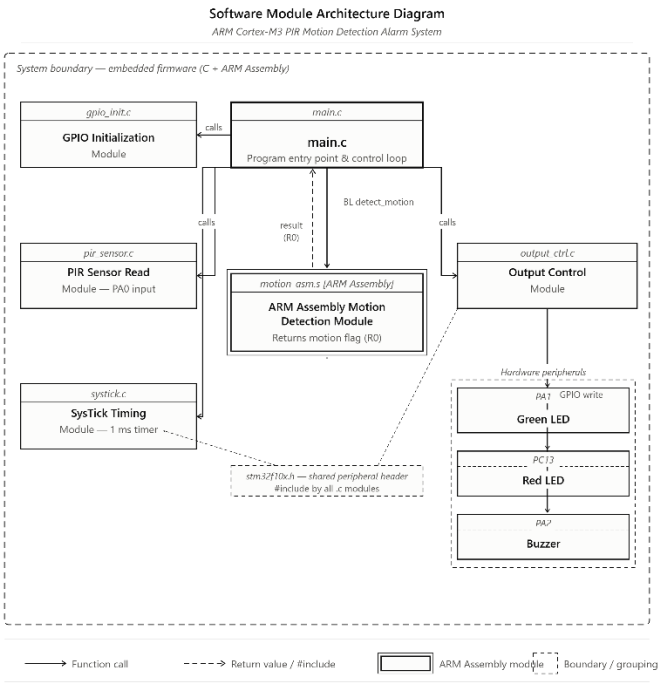


Fig. 3 Software module architecture, PIR Motion Detection Alarm System (ARM Cortex-M3)

3 .معرفی پردازنده انتخابی

در این پروژه برای طراحی و شبیه‌سازی سیستم تشخیص حرکت و اعلام هشدار، از یک میکروکنترلر مبتنی بر معماری ARM Cortex-M3 استفاده شده است. مدل انتخابی برای گزارش، میکروکنترلر **STM32F103C8T6** از خانواده STM32 شرکت STMicroelectronics است. این میکروکنترلر به دلیل داشتن هسته 32 بیتی، امکانات ورودی و خروجی مناسب، تایمر داخلی، پشتیبانی از وقفه‌ها و قابلیت برنامه‌نویسی با زبان C و Assembly ، گزینه مناسبی برای پیاده‌سازی سیستم‌های نهفته ساده و آموزشی محسوب می‌شود.

هسته اصلی این میکروکنترلر، **ARM Cortex-M3**است Cortex-M3 .یک پردازنده 32 بیتی از خانواده ARM است که مخصوص سیستم‌های نهفته طراحی شده است. این هسته برای کاربردهایی مانند کنترل سنسورها، راه‌اندازی خروجی‌ها، پردازش داده‌های ساده و واکنش سریع به رویدادها بسیار مناسب است. در پروژه حاضر نیز پردازنده باید وضعیت سنسور PIR را از ورودی دریافت کند، آن را پردازش نماید و سپس خروجی‌هایی مانند LED و Buzzer را کنترل کند. بنابراین امکانات این پردازنده با نیازهای پروژه هماهنگ است.

فرکانس کاری STM32F103C8T6 می‌تواند تا **72MHz**باشد. این مقدار برای سیستم تشخیص حرکت کاملاً کافی است، زیرا پردازش مورد نیاز پروژه پیچیده نیست و شامل خواندن یک ورودی دیجیتال، بررسی وضعیت آن و کنترل

چند خروجی است. با این حال، استفاده از یک پردازنده Cortex-M3 باعث می‌شود مفاهیم مهم درس مانند رجیسترها، دستورات اسمبلی، Stack، وقفه‌ها و تایمرها به صورت عملی در پروژه دیده شوند.

از نظر حافظه، این میکروکنترلر دارای حافظه **Flash** برای ذخیره کد برنامه و حافظه **SRAM** برای نگهداری داده‌های زمان اجرا است. در این پروژه، کدهای مربوط به فایل C و فایل Assembly در حافظه برنامه قرار می‌گیرند. متغیرهایی مانند motion_detected ، alarm_active، alarm_timer و sys_tick_ms نیز در زمان اجرای برنامه در SRAM نگهداری می‌شوند. این موضوع نشان می‌دهد که برنامه از دو بخش اصلی حافظه استفاده می‌کند: یکی برای ذخیره کد و دیگری برای ذخیره داده‌های موقت و متغیرهای اجرایی.

مهم‌ترین واحد جانبی مورد استفاده در این پروژه، **GPIO** یا پورت‌های ورودی و خروجی عمومی است. سنسور PIR به پایه **PA0** متصل شده و به عنوان ورودی دیجیتال عمل می‌کند. زمانی که حرکت تشخیص داده شود، مقدار این پایه تغییر می‌کند و برنامه آن را بررسی می‌کند. خروجی‌های سیستم نیز شامل LED سبز روی پایه **PA1**، Buzzer روی پایه **PA2** و LED قرمز روی پایه **PC13** هستند. بنابراین در این پروژه از GPIO هم برای دریافت ورودی و هم برای تولید خروجی استفاده شده است.

در کنار GPIO ، از واحد **RCC** نیز برای فعال‌سازی کلاک پورت‌ها استفاده می‌شود. قبل از اینکه برنامه بتواند از پورت‌های GPIOA و GPIOC استفاده کند، باید کلاک مربوط به این پورت‌ها فعال شود. این کار در بخش مقداردهی اولیه برنامه انجام می‌شود و یکی از مراحل مهم راه‌اندازی سخت‌افزار است.

یکی دیگر از امکانات مهم استفاده‌شده در این پروژه، **SysTick Timer** است SysTick .یک تایمر داخلی در پردازنده‌های Cortex-M است که برای ایجاد زمان‌بندی و مدیریت تأخیرهای زمانی استفاده می‌شود. در این پروژه، SysTick برای کنترل مدت فعال بودن هشدار به کار رفته است. به عنوان مثال، پس از تشخیص حرکت، هشدار برای مدت مشخصی فعال می‌ماند و سپس سیستم دوباره به حالت عادی بازمی‌گردد.

پردازنده Cortex-M3 همچنین دارای کنترلر وقفه به نام **NVIC** است NVIC .وظیفه مدیریت وقفه‌ها را بر عهده دارد. در این پروژه، استفاده اصلی از زمان‌بندی SysTick انجام شده است، اما وجود NVIC نشان می‌دهد که سیستم قابلیت توسعه دارد. برای مثال در نسخه‌های کامل‌تر می‌توان تشخیص حرکت سنسور PIR را به جای بررسی مداوم در حلقه اصلی، با وقفه خارجی پیاده‌سازی کرد.

در محیط Keil برای ساده‌سازی شبیه‌سازی، از مدل عمومی **ARMCM3** استفاده شده است. این مدل برای بررسی کدهای C و Assembly و نمایش مفاهیم معماری Cortex-M3 مناسب است. با این حال، آدرس‌دهی رجیسترهای GPIO و ساختار کلی پروژه بر اساس خانواده STM32F103 در نظر گرفته شده است. بنابراین در گزارش، پردازنده انتخابی STM32F103C8T6 معرفی می‌شود و محیط شبیه‌سازی به عنوان Keil uVision با مدل ARM Cortex-M3 توضیح داده می‌شود.

در مجموع، STM32F103C8T6 برای این پروژه انتخاب مناسبی است، زیرا امکانات لازم برای خواندن سنسور PIR، پردازش وضعیت حرکت، کنترل LED ها و Buzzer ، استفاده از تایمر و پیاده‌سازی بخشی از برنامه در Assembly را فراهم می‌کند. به همین دلیل این پردازنده هم از نظر فنی و هم از نظر آموزشی با هدف پروژه هماهنگ است.

4.طراحی نرم‌افزار در زبان C

در این پروژه، بخش اصلی نرم‌افزار سیستم تشخیص حرکت و اعلام هشدار در زبان C طراحی شده است. زبان C به دلیل خوانایی مناسب، امکان تعریف توابع ساختاریافته و قابلیت دسترسی مستقیم به رجیسترهای سخت‌افزاری، یکی از زبان‌های پرکاربرد در برنامه‌نویسی سیستم‌های نهفته محسوب می‌شود. در این پروژه، زبان C وظیفه مدیریت کلی سیستم، مقداردهی اولیه پورت‌ها، کنترل خروجی‌ها، مدیریت زمان و ارتباط با روتین اسمبلی را بر عهده دارد.

برنامه C پروژه در فایل main.c قرار دارد. این فایل به عنوان هسته اصلی نرم‌افزار عمل می‌کند و اجرای کل سیستم از تابع main آغاز می‌شود. در ابتدای برنامه، آدرس رجیسترهای سخت‌افزاری مورد نیاز مانند GPIO و RCC تعریف شده‌اند. این رجیسترها برای راه‌اندازی و کنترل پورت‌های ورودی و خروجی استفاده می‌شوند. در معماری Cortex-M، رجیسترهای سخت‌افزاری به صورت Memory-Mapped در فضای حافظه قرار دارند؛ بنابراین برنامه می‌تواند با خواندن و نوشتن در آدرس‌های مشخص، وضعیت پایه‌ها را کنترل کند.
در این پروژه، پایه PA0 به عنوان ورودی سنسور PIR در نظر گرفته شده است. این سنسور وضعیت حرکت را به صورت سیگنال دیجیتال به میکروکنترلر ارسال می‌کند. اگر حرکتی وجود نداشته باشد، مقدار ورودی صفر است و اگر حرکت تشخیص داده شود، مقدار ورودی یک می‌شود. خروجی‌های سیستم نیز شامل LED سبز روی پایهPA1 ، Buzzerروی پایه PA2 و LED قرمز روی پایه PC13 هستند .LED سبز نشان‌دهنده حالت عادی سیستم است و LEDقرمز و Buzzer برای حالت هشدار استفاده می‌شوند.

در ابتدای اجرای برنامه، تابع مقداردهی اولیه GPIO اجرا می‌شود. این تابع ابتدا کلاک پورت‌های مورد نیاز را از طریق RCC فعال می‌کند. سپس پایه PA0 را به عنوان ورودی و پایه‌هایPA1 ، PA2و PC13 را به عنوان خروجی تنظیم می‌کند. این مرحله اهمیت زیادی دارد، زیرا قبل از استفاده از هر پایه، باید نوع عملکرد آن مشخص شود. اگر پایه‌ها به درستی مقداردهی اولیه نشوند، سیستم قادر به خواندن سنسور یا کنترل خروجی‌ها نخواهد بود.

پس از راه‌اندازی پایه‌ها، تایمر SysTick نیز مقداردهی اولیه می‌شود SysTick .یک تایمر داخلی در پردازنده‌های Cortex-Mاست که برای ایجاد زمان‌بندی استفاده می‌شود. در این پروژه، از SysTick برای کنترل مدت فعال بودن هشدار استفاده شده است. زمانی که حرکت تشخیص داده می‌شود، مقدار تایمر هشدار تنظیم می‌شود و سپس در هر بار اجرای وقفه SysTick کاهش می‌یابد. با رسیدن تایمر به صفر، سیستم از حالت هشدار خارج شده و به حالت عادی برمی‌گردد.

بعد از مقداردهی اولیه، برنامه وارد حلقه اصلی می‌شود. در سیستم‌های نهفته، معمولاً برنامه اصلی در یک حلقه بی‌نهایت اجرا می‌شود، زیرا سیستم باید به صورت مداوم وضعیت محیط را بررسی کند. در این پروژه نیز برنامه در حلقه اصلی، وضعیت سنسور PIR را بررسی می‌کند. برای این کار، تابع اسمبلی ASM_CheckMotion از داخل برنامه C فراخوانی می‌شود. این تابع مقدار سنسور را بررسی کرده و نتیجه تشخیص حرکت را در اختیار برنامه C قرار می‌دهد.

ارتباط بین C و Assembly یکی از بخش‌های مهم پروژه است. در فایلC ، تابع اسمبلی با استفاده از extern معرفی می‌شود تا کامپایلر بداند پیاده‌سازی آن در فایل دیگری قرار دارد. از طرف دیگر، فایل اسمبلی نتیجه تشخیص حرکت را در متغیر مشترک motion_detected ذخیره می‌کند. برنامه C این متغیر را بررسی می‌کند و اگر مقدار آن نشان‌دهنده تشخیص حرکت باشد، حالت هشدار فعال می‌شود.

برای مدیریت وضعیت سیستم، چند متغیر اصلی در برنامه تعریف شده‌اند. متغیر motion_detected مشخص می‌کند که آیا حرکت تشخیص داده شده است یا خیر. متغیر alarm_active نشان می‌دهد که سیستم در حالت هشدار قرار دارد یا نه. متغیر alarm_timer نیز مدت زمان باقی‌مانده برای فعال بودن هشدار را نگهداری می‌کند. این متغیرها به صورت volatile تعریف شده‌اند، زیرا ممکن است مقدار آن‌ها در بخش‌هایی مانند وقفه SysTick یا روتین اسمبلی تغییر کند.

زمانی که حرکت تشخیص داده شود، تابع فعال‌سازی هشدار اجرا می‌شود. در این حالت، LEDسبز خاموش شده و LEDقرمز و Buzzer فعال می‌شوند. همچنین تایمر هشدار مقداردهی می‌شود تا هشدار برای مدت مشخصی فعال باقی بماند. پس از پایان این زمان، تابع غیرفعال‌سازی هشدار اجرا می‌شود. در این حالت، LEDقرمز و Buzzer خاموش شده و LED سبز دوباره روشن می‌شود.

در مجموع، طراحی نرم‌افزار در زبان C به گونه‌ای انجام شده است که برنامه ساختاریافته، قابل فهم و قابل توسعه

باشد. زبان C وظیفه مدیریت سطح بالای سیستم را بر عهده دارد و بخش اسمبلی برای پردازش مستقیم‌تر وضعیت سنسور استفاده شده است. این ترکیب باعث می‌شود پروژه هم از نظر عملکردی کامل باشد و هم ارتباط مناسبی با مباحث درس ریزپردازنده و زبان اسمبلی داشته باشد.

5 .پیاده‌سازی بخش‌هایAssembly ARM

در این پروژه، علاوه بر برنامه‌نویسی اصلی در زبانC ، بخشی از عملیات پردازش وضعیت سنسور PIR در زبان Assembly ARMپیاده‌سازی شده است. هدف از استفاده از اسمبلی این است که ارتباط پروژه با مباحث درس ریزپردازنده و زبان اسمبلی به صورت عملی نشان داده شود. زبان C برای مدیریت کلی سیستم، مقداردهی اولیه پایه‌ها، کنترل خروجی‌ها و مدیریت زمان مناسب است؛ اما زبان Assembly امکان کنترل دقیق‌تر روی رجیسترهای پردازنده، دستورات سطح پایین و نحوه ارتباط مستقیم با سخت‌افزار را فراهم می‌کند.

فایل اسمبلی پروژه با نام motion_asm.s در پروژه قرار گرفته است. این فایل شامل چند زیرروال اصلی است که برای خواندن وضعیت پایه ورودی، نوشتن روی خروجی، ایجاد تأخیر نرم‌افزاری و بررسی وضعیت حرکت استفاده می‌شوند. مهم‌ترین زیرروال این فایل، تابع ASM_CheckMotion است. این تابع وظیفه دارد وضعیت سنسور PIR متصل به پایه PA0 را بررسی کند و در صورت تشخیص حرکت، نتیجه را در متغیر motion_detected ذخیره نماید تا برنامه C بتواند بر اساس آن خروجی‌های هشدار را کنترل کند.

در ابتدای فایل اسمبلی، بخش‌های مربوط به داده و کد مشخص شده‌اند. بخش داده برای تعریف متغیرها و بخش کد برای نوشتن دستورهای اجرایی استفاده می‌شود. همچنین در این فایل از دستور THUMB استفاده شده است، زیرا پردازنده‌های خانواده Cortex-M مانند Cortex-M3 از مجموعه دستورهای Thumb و Thumb-2 استفاده می‌کنند. این مجموعه دستورها برای سیستم‌های نهفته مناسب هستند، زیرا هم حجم کد را کاهش می‌دهند و هم امکان اجرای دستورهای لازم برای کنترل سخت‌افزار را فراهم می‌کنند.

در فایل اسمبلی، آدرس برخی رجیسترهای سخت‌افزاری با استفاده از دستور EQU تعریف شده‌اند. برای مثال، رجیستر GPIOA_IDR برای خواندن وضعیت پایه‌های ورودی پورت A استفاده می‌شود. همچنین رجیسترهای خروجی مانند GPIOA_ODR و GPIOC_ODR برای کنترل پایه‌های خروجی پورت‌های A و C تعریف شده‌اند. این موضوع نشان می‌دهد که در معماریCortex-M ، رجیسترهای سخت‌افزاری به صورت Memory-Mapped در فضای حافظه قرار دارند. یعنی برنامه می‌تواند با خواندن یا نوشتن در یک آدرس مشخص، مستقیماً وضعیت سخت‌افزار را کنترل کند.

برای اینکه توابع اسمبلی از داخل برنامه C قابل فراخوانی باشند، در فایل اسمبلی از دستور EXPORT استفاده شده است. توابعی مانندASM_GPIO_ReadPin ، ASM_GPIO_WritePin، ASM_Delay_Loopو ASM_CheckMotionبه این شکل معرفی شده‌اند. از طرف دیگر، متغیر motion_detected که در فایل C تعریف شده، در فایل اسمبلی با دستور IMPORT معرفی شده است. به این ترتیب، کد اسمبلی می‌تواند مقدار این متغیر را تغییر دهد و نتیجه تشخیص حرکت را به برنامه C منتقل کند.

زیرروال ASM_GPIO_ReadPin برای خواندن وضعیت یک پایه از پورت GPIOA استفاده می‌شود. در این تابع، شماره پایه مورد نظر از طریق رجیستر R0 دریافت می‌شود. طبق قرارداد فراخوانی درARM ، رجیستر R0 معمولاً برای دریافت اولین آرگومان تابع و همچنین بازگرداندن مقدار خروجی استفاده می‌شود. در این زیرروال ابتدا آدرس رجیستر ورودی پورت A در یکی از رجیسترها قرار می‌گیرد، سپس مقدار این رجیستر با دستور LDR خوانده می‌شود. پس از آن با استفاده از یک ماسک بیتی، فقط وضعیت پایه مورد نظر بررسی می‌شود.

برای ساخت ماسک بیتی، ابتدا عدد 1 در یک رجیستر قرار داده می‌شود و سپس با توجه به شماره پایه، به سمت چپ

شیفت داده می‌شود. سپس با دستور AND مقدار رجیستر ورودی و ماسک ساخته‌شده با هم ترکیب می‌شوند. اگر نتیجه صفر باشد، یعنی پایه مورد نظر مقدار صفر دارد و اگر نتیجه غیرصفر باشد، یعنی پایه مقدار یک دارد. در پایان، نتیجه به صورت صفر یا یک در رجیستر R0 قرار داده می‌شود و تابع به محل فراخوانی بازمی‌گردد. این بخش نشان می‌دهد که چگونه می‌توان با عملیات بیتی ساده، وضعیت یک پایه خاص را از بین چند پایه یک پورت استخراج کرد.
زیرروال ASM_GPIO_WritePin برای تغییر وضعیت پایه‌های خروجی استفاده می‌شود. این تابع معمولاً آدرس رجیستر خروجی، ماسک پایه و مقدار مورد نظر را دریافت می‌کند. اگر مقدار ورودی برابر یک باشد، بیت مربوط به پایه خروجی فعال می‌شود و اگر مقدار ورودی صفر باشد، آن بیت پاک می‌شود. برای فعال کردن بیت از دستور منطقی ORR استفاده می‌شود. برای پاک کردن بیت نیز ابتدا ماسک معکوس ساخته شده و سپس با دستور AND بیت مورد نظر صفر می‌شود. در نهایت مقدار جدید با دستور STR در رجیستر خروجی نوشته می‌شود.
استفاده از این روش اهمیت زیادی دارد، زیرا در رجیسترهای GPIO هر بیت معمولاً مربوط به یک پایه جداگانه است. بنابراین هنگام روشن یا خاموش کردن یک پایه، نباید وضعیت سایر پایه‌ها تغییر کند. به همین دلیل، در اسمبلی از عملیات منطقی برای تغییر کنترل‌شده فقط یک بیت استفاده شده است.

این موضوع یکی از کاربردهای مهم دستورات منطقی در برنامه‌نویسی سطح پایین و سیستم‌های نهفته است.
زیرروال دیگر فایل اسمبلی، ASM_Delay_Loopاست. این تابع برای ایجاد یک تأخیر نرم‌افزاری ساده استفاده می‌شود. مقدار اولیه شمارنده از طریق رجیستر R0 وارد تابع می‌شود. سپس در یک حلقه، مقدار این رجیستر با دستور SUBS کاهش می‌یابد. دستور SUBS علاوه بر انجام عمل تفریق، پرچم‌های وضعیت پردازنده را نیز تغییر می‌دهد. بعد از آن، دستور BNE بررسی می‌کند که آیا نتیجه هنوز صفر نشده است یا نه. اگر مقدار صفر نشده باشد، برنامه دوباره به ابتدای حلقه برمی‌گردد. وقتی مقدار شمارنده به صفر برسد، تابع پایان می‌یابد.
این تأخیر نرم‌افزاری در پروژه برای ایجاد فاصله کوتاه بین دو بار خواندن وضعیت سنسور PIR استفاده شده است. دلیل این کار آن است که گاهی ممکن است ورودی سنسور به صورت لحظه‌ای تغییر کند یا ناپایدار باشد. بنابراین برنامه پس از تشخیص اولیه حرکت، کمی صبر می‌کند و سپس دوباره سنسور را بررسی می‌کند. اگر در خواندن دوم نیز مقدار سنسور نشان‌دهنده حرکت باشد، حرکت تأیید می‌شود. این کار نوعی Debounce یا تأیید نرم‌افزاری ساده برای کاهش خطای ورودی است.
مهم‌ترین تابع اسمبلی پروژه ASM_CheckMotion است. این تابع از داخل حلقه اصلی برنامه C فراخوانی می‌شود. ابتدا در این تابع مقدار صفر در رجیستر R0 قرار می‌گیرد، زیرا سنسور PIR به پایه PA0 متصل است. سپس تابع ASM_GPIO_ReadPinبا دستور BL فراخوانی می‌شود. دستور BL یا Branch with Link برای فراخوانی زیرروال استفاده می‌شود و آدرس بازگشت را در رجیستر LR ذخیره می‌کند. پس از برگشت از تابع خواندن پایه، مقدار سنسور در رجیستر R0 قرار دارد.
در ادامه، مقدار خوانده‌شده با عدد یک مقایسه می‌شود. این کار با دستور CMP انجام می‌گیرد. اگر مقدار سنسور برابر یک نباشد، یعنی حرکتی تشخیص داده نشده است و برنامه با دستور پرش شرطی BNE به بخش بدون حرکت منتقل می‌شود. در این حالت مقدار متغیر motion_detected صفر قرار داده می‌شود. اما اگر مقدار سنسور یک باشد، برنامه وارد مرحله تأیید حرکت می‌شود.
در مرحله تأیید حرکت، ابتدا یک تأخیر کوتاه با استفاده از ASM_Delay_Loop اجرا می‌شود. سپس وضعیت سنسور دوباره خوانده می‌شود. اگر مقدار سنسور در خواندن دوم نیز برابر یک باشد، حرکت تأیید شده و مقدار متغیر motion_detectedبرابر یک قرار می‌گیرد. برای نوشتن مقدار در این متغیر از دستور STRB استفاده می‌شود، زیرا متغیر مورد نظر از نوع بایتی است. اگر در خواندن دوم مقدار صفر باشد، برنامه نتیجه را به عنوان عدم تشخیص حرکت در نظر می‌گیرد و مقدار motion_detected صفر می‌شود.
در این فایل اسمبلی، از چند دسته دستور مهم استفاده شده است. دستورهای LDR و STR برای خواندن و نوشتن داده از حافظه یا رجیسترهای سخت‌افزاری به کار رفته‌اند. دستور STRB برای نوشتن یک بایت در حافظه استفاده شده است. دستورهای منطقی مانندAND ، ORRو MVN برای بررسی یا تغییر بیت‌های خاص در رجیسترهای GPIOاستفاده شده‌اند. دستور CMP برای مقایسه مقدار سنسور با عدد یک استفاده شده و دستورهای پرش شرطی مانند BEQ و BNE مسیر اجرای برنامه را بر اساس نتیجه مقایسه تغییر داده‌اند.
همچنین در زیرروال‌های اسمبلی از دستورات PUSH و POP برای مدیریت Stack استفاده شده است. زمانی که یک زیرروال اجرا می‌شود، ممکن است مقدار برخی رجیسترها در طول اجرای آن تغییر کند. برای جلوگیری از خراب شدن وضعیت برنامه، این رجیسترها در ابتدای تابع روی Stack ذخیره شده و در پایان تابع بازیابی می‌شوند. رجیستر

LRنیز معمولاً روی Stack ذخیره می‌شود تا آدرس بازگشت از زیرروال حفظ شود. در پایان برخی توابع، با دستور POP {PC}برنامه به محل فراخوانی بازمی‌گردد.

از نظر مدیریت رجیسترها، رجیسترهای R0 تا R3 بیشتر برای ارسال آرگومان، دریافت مقدار برگشتی و نگهداری مقادیر موقت استفاده شده‌اند. رجیستر R0 در پروژه نقش اصلی دارد، زیرا هم شماره پایه ورودی را به تابع می‌دهد و هم نتیجه خواندن پایه یا نتیجه نهایی تابع را برمی‌گرداند. رجیسترهایR1 ، R2و R3 برای نگهداری آدرس رجیسترها، ماسک بیتی و مقادیر واسط استفاده شده‌اند. رجیستر LR نیز آدرس بازگشت از زیرروال‌ها را نگهداری می‌کند.
در مجموع، بخش Assembly ARM در این پروژه نقش مهمی دارد. این بخش نشان می‌دهد که چگونه می‌توان با استفاده از دستورات سطح پایین، وضعیت یک سنسور را از طریق رجیسترهای GPIO خواند، آن را پردازش کرد و نتیجه را در اختیار برنامه C قرار داد. تابع ASM_CheckMotion بخش اصلی تصمیم‌گیری مربوط به تشخیص حرکت را انجام می‌دهد و برنامه C بر اساس نتیجه آن، LEDسبز، LEDقرمز و Buzzer را کنترل می‌کند.بنابراین این بخش هم از نظر عملکردی در پروژه مؤثر است و هم از نظر آموزشی، مفاهیم اصلی درس ریزپردازنده و زبان اسمبلی را به صورت عملی نمایش می‌دهد.

6 .تحلیل معماری و ارتباط با مباحث درس

در این پروژه از معماری **ARM Cortex-M3**برای طراحی و شبیه‌سازی سیستم تشخیص حرکت و اعلام هشدار استفاده شده است. این معماری برای سیستم‌های نهفته طراحی شده و در کاربردهایی مانند کنترل سنسورها، مدیریت ورودی و خروجی، اجرای برنامه‌های کنترلی و واکنش به رویدادها کاربرد زیادی دارد. پروژه حاضر نیز یک سیستم نهفته ساده است که ورودی آن سنسور PIR و خروجی‌های آن LED سبز، LEDقرمز و Buzzer هستند. بنابراین ساختار کلی پروژه با ویژگی‌های معماری Cortex-M3 هماهنگ است.
پردازنده Cortex-M3 یک پردازنده 32 بیتی است. این یعنی رجیسترهای عمومی آن معمولاً 32 بیت عرض دارند و می‌توانند داده‌ها و آدرس‌ها را به صورت 32 بیتی پردازش کنند. در این پروژه، این ویژگی در بخش اسمبلی و همچنین در دسترسی به آدرس رجیسترهای سخت‌افزاری اهمیت دارد. برای مثال، رجیسترهای مربوط به GPIO و RCC در آدرس‌های مشخصی از حافظه قرار دارند و برنامه با استفاده از این آدرس‌ها می‌تواند مستقیماً پایه‌های ورودی و خروجی را کنترل کند.
یکی از مفاهیم مهم در معماریARM ، معماری **Load/Store**است. در این معماری، عملیات پردازشی به طور مستقیم روی حافظه انجام نمی‌شود، بلکه ابتدا داده باید از حافظه یا رجیستر سخت‌افزاری به رجیسترهای داخلی پردازنده منتقل شود. سپس عملیات پردازشی روی رجیستر انجام می‌گیرد و در صورت نیاز، نتیجه دوباره در حافظه یا رجیستر سخت‌افزاری نوشته می‌شود. در فایل اسمبلی پروژه، این مفهوم با دستورهای LDR و STR دیده می‌شود. دستور LDR برای خواندن مقدار از حافظه یا رجیستر سخت‌افزاری و دستور STR برای نوشتن مقدار در حافظه یا رجیستر خروجی استفاده شده است.
در پروژه حاضر، سنسور PIR به پایه PA0 متصل شده است. وضعیت این پایه از طریق رجیستر ورودی پورت A خوانده می‌شود. در بخش اسمبلی، ابتدا آدرس رجیستر ورودی GPIOA در یک رجیستر قرار می‌گیرد، سپس مقدار آن با دستور LDR خوانده می‌شود. بعد با استفاده از عملیات بیتی، فقط بیت مربوط به پایه PA0 بررسی می‌گردد. اگر این بیت برابر یک باشد، یعنی سنسور حرکت را تشخیص داده است و اگر صفر باشد، یعنی حرکتی تشخیص داده نشده است. این فرایند نشان می‌دهد که مفاهیم رجیستر، آدرس‌دهی حافظه و عملیات منطقی مستقیماً در عملکرد پروژه نقش دارند.
در معماریCortex-M ، بسیاری از واحدهای جانبی مانندGPIO ، RCC، SysTickو تایمرها به صورت **Memory-Mapped**در فضای آدرس‌دهی پردازنده قرار دارند. یعنی هر رجیستر سخت‌افزاری مانند یک خانه حافظه دارای آدرس مشخص است. برنامه می‌تواند با خواندن یا نوشتن در این آدرس‌ها، سخت‌افزار را کنترل کند. در این پروژه، برای فعال‌سازی کلاک پورت‌ها از RCC استفاده شده و برای کنترل پایه‌ها از رجیسترهای GPIO استفاده می‌شود. پایه PA0 به عنوان ورودی، پایه PA1 برای LED سبز، پایه PA2 برای Buzzer و پایه PC13 برای LEDقرمز در نظر گرفته شده‌اند.
از نظر حافظه، برنامه از دو بخش اصلی استفاده می‌کند: حافظه **Flash**و حافظه **.SRAM**کدهای برنامه شامل فایل

Cو فایل Assembly پس از Build شدن در حافظه Flash قرار می‌گیرند. این بخش حافظه برای ذخیره برنامه استفاده می‌شود. در مقابل، متغیرهایی که هنگام اجرای برنامه تغییر می‌کنند، در SRAM ذخیره می‌شوند. برای مثال، متغیرهایmotion_detected ، alarm_active، alarm_timerو sys_tick_ms در زمان اجرا مقداردهی و تغییر داده می‌شوند. بنابراین این متغیرها در حافظه داده قرار می‌گیرند و برنامه در طول اجرا از آن‌ها برای مدیریت وضعیت سیستم استفاده می‌کند.

رجیسترها نیز در اجرای پروژه نقش اصلی دارند. در معماریARM ، رجیسترهای عمومی R0 تا R12 برای انجام عملیات مختلف استفاده می‌شوند. در پروژه حاضر، رجیستر R0 اهمیت زیادی دارد، زیرا در توابع اسمبلی هم برای دریافت ورودی و هم برای بازگرداندن نتیجه به کار می‌رود. برای مثال، در تابع ASM_GPIO_ReadPin شماره پایه از طریق R0 دریافت می‌شود و نتیجه خواندن پایه نیز از طریق همین رجیستر بازگردانده می‌شود. رجیسترهای R1، R2و R3 نیز برای نگهداری آدرس رجیسترها، مقدار خوانده‌شده، ماسک بیتی و مقادیر موقت استفاده شده‌اند. رجیستر LR یا Link Register برای نگهداری آدرس بازگشت از زیرروال استفاده می‌شود. زمانی که در اسمبلی از دستور BL برای فراخوانی یک تابع استفاده می‌شود، آدرس دستور بعدی در LR ذخیره می‌گردد. سپس پس از پایان زیرروال، برنامه می‌تواند به محل فراخوانی بازگردد. در پروژه حاضر، دستور BL برای فراخوانی توابعی مانند ASM_GPIO_ReadPinو ASM_Delay_Loop استفاده شده است. این موضوع نشان می‌دهد که فراخوانی زیرروال‌ها در سطح اسمبلی چگونه انجام می‌شود.
یکی دیگر از مفاهیم مهم معماری، **Stack**و رجیستر **Stack Pointer**است.

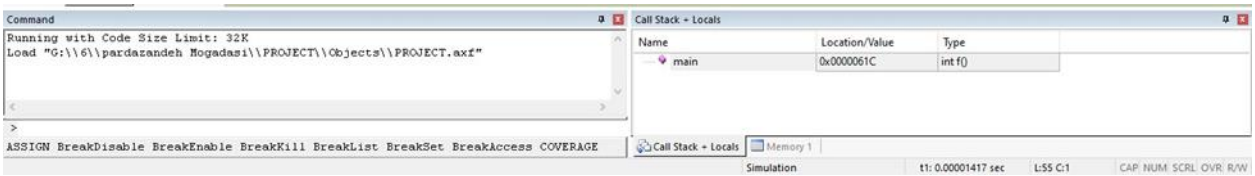
Stackبخشی از حافظه RAM است که برای ذخیره موقت داده‌ها، رجیسترها و آدرس بازگشت از توابع استفاده می‌شود. در فایل اسمبلی پروژه، از دستورهای PUSH و POP برای کار با Stack استفاده شده است. وقتی یک زیرروال شروع می‌شود، برخی رجیسترها روی Stack ذخیره می‌شوند تا مقدار قبلی آن‌ها از بین نرود. در پایان زیرروال، همان مقادیر با دستور POP بازیابی می‌شوند. این کار باعث می‌شود اجرای توابع اسمبلی باعث خراب شدن وضعیت برنامه C یا زیرروال‌های دیگر نشود.
معماری Cortex-M3 همچنین از ساختار **Pipeline**استفاده می‌کند Pipeline .باعث می‌شود مراحل اجرای دستورها مانند دریافت دستور، رمزگشایی و اجرا به صورت همپوشان انجام شوند. به زبان ساده، پردازنده لازم نیست برای شروع دستور بعدی منتظر پایان کامل دستور قبلی بماند. این موضوع باعث افزایش سرعت اجرای برنامه می‌شود. در پروژه حاضر، دستورهای پشت سر هم مانندLDR ، CMP، ANDو BNE توسط Pipeline پردازنده مدیریت می‌شوند. البته دستورهای پرش شرطی مانند BEQ و BNE می‌توانند مسیر اجرای برنامه را تغییر دهند و روی Pipelineاثر بگذارند، زیرا پردازنده باید بر اساس نتیجه شرط مسیر بعدی اجرا را مشخص کند.
در این پروژه از مفهوم وقفه نیز از طریق **SysTick**استفاده شده است SysTick .یک تایمر داخلی در پردازنده‌های Cortex-Mاست که می‌تواند در بازه‌های زمانی مشخص وقفه ایجاد کند. در برنامهC ، تابع SysTick_Handler وظیفه رسیدگی به این وقفه را بر عهده دارد. در هر بار اجرای این تابع، شمارنده زمان افزایش می‌یابد و اگر هشدار فعال باشد، مقدار alarm_timer کاهش پیدا می‌کند. این روش باعث می‌شود سیستم بتواند مدت فعال بودن هشدار را کنترل کند، بدون اینکه برنامه اصلی به طور کامل متوقف شود.
وجود کنترلر وقفه **NVIC**در Cortex-M3 نیز اهمیت دارد NVIC .وظیفه مدیریت وقفه‌ها را بر عهده دارد و امکان پاسخ‌گویی سریع پردازنده به رویدادها را فراهم می‌کند. در این پروژه استفاده اصلی از وقفه مربوط به SysTick است، اما ساختار سیستم قابلیت توسعه دارد. برای مثال، در نسخه کامل‌تر می‌توان ورودی سنسور PIR را به وقفه خارجی متصل کرد تا پردازنده فقط هنگام تغییر وضعیت سنسور واکنش نشان دهد.
در مجموع، این پروژه نمونه‌ای ساده اما کامل از کاربرد مفاهیم معماری در یک سیستم نهفته است. برنامه از Flash اجرا می‌شود، داده‌های زمان اجرا در SRAM قرار می‌گیرند، رجیسترهای GPIO به صورت Memory-Mappedکنترل می‌شوند، از Stack برای حفظ رجیسترها در زیرروال‌های Assembly استفاده می‌شود و SysTickبرای مدیریت زمان هشدار به کار می‌رود. بنابراین پروژه ارتباط مستقیمی با مباحث درس ریزپردازنده و زبان اسمبلی دارد و نشان می‌دهد که مفاهیم معماری پردازنده چگونه در طراحی و اجرای یک سیستم واقعی به کار گرفته می‌شوند.

7 . شبیه‌سازی و نتیجه

پروژه سیستم تشخیص حرکت و اعلام هشدار با سنسور PIR در محیط **Keil uVision** پیاده‌سازی و شبیه‌سازی شده است Keil .یکی از محیط‌های مناسب برای برنامه‌نویسی و بررسی پروژه‌های مبتنی بر پردازنده‌های ARM است و امکان اضافه کردن فایل‌های C و Assembly ، Build گرفتن از پروژه، مشاهده خطاها و بررسی عملکرد برنامه در حالت Debug را فراهم می‌کند. در این پروژه، فایل اصلی برنامه با نام main.c و فایل اسمبلی با نام motion_asm.s به پروژه اضافه شده‌اند و اجرای کلی سیستم با ترکیب این دو بخش انجام می‌شود.

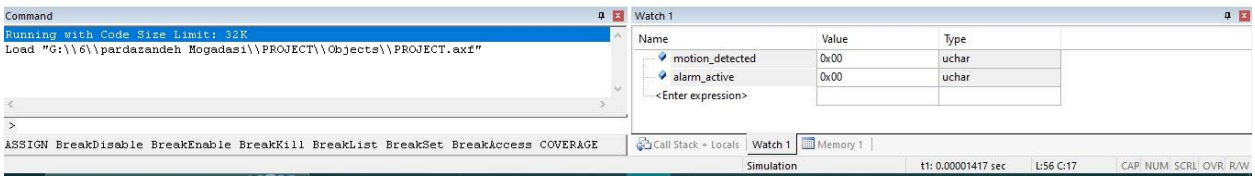
در مرحله پیاده‌سازی، ابتدا فایل main.c وظیفه مقداردهی اولیه پورت‌ها، تنظیم ورودی و خروجی‌ها، مدیریت حلقه اصلی برنامه، کنترل LED ها و Buzzer و استفاده از SysTick را بر عهده دارد. در کنار آن، فایل motion_asm.s شامل روتین‌های Assembly برای خواندن وضعیت سنسور PIR ، بررسی مقدار ورودی، ایجاد تأخیر نرم‌افزاری و تعیین وضعیت حرکت است. این ترکیب باعث می‌شود پروژه هم از ساختار منظم زبان C استفاده کند و هم بخش مهمی از پردازش را در سطح پایین و با زبان Assembly انجام دهد.

پس از تکمیل کدنویسی، پروژه در محیط Keil Build شد. نتیجه Build نشان داد که فایل‌های C و Assembly بدون خطا در کنار یکدیگر کامپایل و لینک شده‌اند. این موضوع اهمیت زیادی دارد، زیرا در پروژه‌هایی که C و Assembly به صورت هم‌زمان استفاده می‌شوند، ممکن است خطاهایی مانند شناخته نشدن تابع اسمبلی، اشتباه در نام تابع، خطای Linker یا مشکل در ارتباط متغیرهای مشترک ایجاد شود. موفق بودن Build نشان می‌دهد که ارتباط بین main.c و motion_asm.s به درستی برقرار شده است.

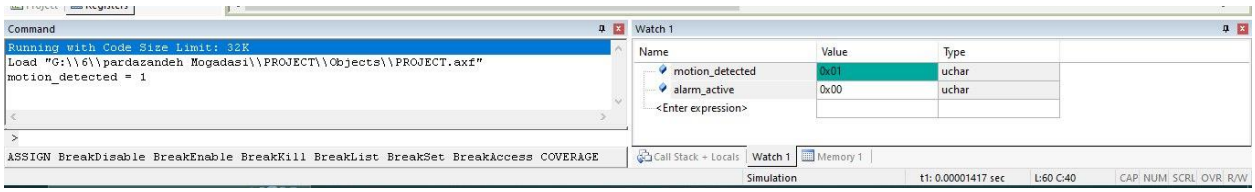


برای شبیه‌سازی عملکرد سیستم، دو حالت اصلی بررسی شد. حالت اول، وضعیت عادی سیستم است. در این حالت فرض می‌شود که سنسور PIR هیچ حرکتی را تشخیص نداده و مقدار ورودی پایه PA0 برابر صفر است. در این وضعیت، برنامه باید سیستم را در حالت Normal نگه دارد. بنابراین LED سبز روشن می‌شود و LED قرمز و Buzzer خاموش باقی می‌مانند. این حالت نشان‌دهنده آماده بودن سیستم و نبود وضعیت هشدار است

.



حالت دوم، وضعیت تشخیص حرکت است. در این حالت فرض می‌شود که سنسور PIR حرکت را تشخیص داده و مقدار پایه PA0 برابر یک شده است. برنامه اصلی در حلقه اجرای خود، تابع اسمبلی ASM_CheckMotion را فراخوانی می‌کند. این تابع مقدار ورودی را بررسی کرده و در صورت تأیید حرکت، مقدار متغیر motion_detected را برابر یک قرار می‌دهد. سپس برنامه C این مقدار را بررسی می‌کند و اگر حرکت تشخیص داده شده باشد، سیستم وارد حالت هشدار می‌شود. در این وضعیت، LED سبز خاموش شده و LED قرمز و Buzzer فعال می‌شوند.

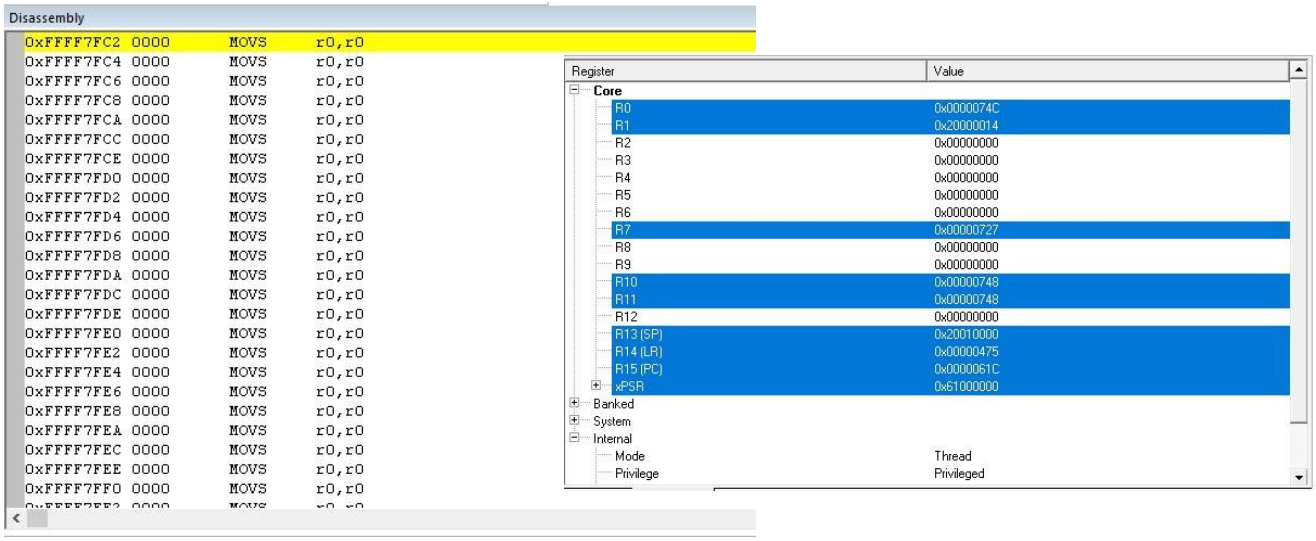


یکی از بخش‌های مهم شبیه‌سازی، بررسی زمان‌بندی هشدار است. در پروژه، پس از تشخیص حرکت، متغیر alarm_timerمقداردهی می‌شود تا هشدار برای مدت مشخصی فعال باقی بماند. این مقدار با استفاده از وقفه SysTickکاهش پیدا می‌کند. تا زمانی که مقدار alarm_timer بیشتر از صفر باشد، سیستم در حالت هشدار باقی می‌ماند. پس از رسیدن مقدار تایمر به صفر، خروجی‌های هشدار غیرفعال می‌شوند و سیستم دوباره به حالت عادی بازمی‌گردد. در این حالت، LEDقرمز و Buzzer خاموش شده و LED سبز روشن می‌شود.

در زمان شبیه‌سازی، متغیرهای مهمی مانندmotion_detected ، alarm_activeو alarm_timer قابل بررسی هستند. تغییر مقدار این متغیرها نشان می‌دهد که منطق برنامه به درستی عمل می‌کند. برای مثال، پس از تشخیص حرکت، مقدار motion_detected و alarm_active برابر یک می‌شود و مقدار alarm_timer شروع به کاهش می‌کند. پس از پایان زمان هشدار، مقدار alarm_active صفر شده و خروجی‌ها به وضعیت عادی برمی‌گردند.

نتایج شبیه‌سازی نشان می‌دهد که سیستم طراحی‌شده عملکرد مورد انتظار را دارد. سیستم می‌تواند ورودی سنسور PIRرا بررسی کند، وضعیت حرکت را با کمک روتین Assembly تشخیص دهد و خروجی‌های مناسب را از طریق برنامه C کنترل کند. همچنین استفاده از SysTick باعث شده است که مدت زمان فعال بودن هشدار قابل مدیریت باشد. بنابراین پروژه از نظر دریافت ورودی، پردازش داده، کنترل خروجی و ترکیب زبان C با Assembly عملکرد قابل قبولی دارد.

در مجموع، شبیه‌سازی پروژه نشان داد که ساختار طراحی‌شده برای یک سیستم نهفته ساده مناسب است. پروژه توانسته است سه بخش اصلی یک سیستم Embedded یعنی ورودی، پردازش و خروجی را پیاده‌سازی کند. ورودی سیستم سنسورPIR ، بخش پردازش شامل میکروکنترلر ARM Cortex-M3 و کدهای C وAssembly ، و خروجی سیستم شامل LED سبز، LEDقرمز و Buzzer است. این نتیجه نشان می‌دهد که پروژه با هدف درس ریزپردازنده و زبان اسمبلی هماهنگ بوده و می‌تواند به عنوان نمونه‌ای آموزشی از طراحی و شبیه‌سازی یک سیستم نهفته مورد استفاده قرار گیرد .



8 . جمع‌بندی و منابع

8-1 . جمع‌بندی

در این پروژه، یک سیستم نهفته ساده برای تشخیص حرکت و اعلام هشدار طراحی و شبیه‌سازی شد. هدف اصلی پروژه این بود که یک سیستم دارای ورودی مشخص، پردازش مشخص و خروجی مشخص طراحی شود و ارتباط آن با مباحث درس ریزپردازنده و زبان اسمبلی نشان داده شود. برای این منظور، سنسور PIR به عنوان ورودی سیستم انتخاب شد. این سنسور وظیفه تشخیص حرکت در محیط را بر عهده دارد و خروجی آن به پایه PA0 میکروکنترلر متصل شده است.

پردازش اصلی سیستم توسط میکروکنترلر مبتنی بر معماری ARM Cortex-M3 انجام شده است. برنامه اصلی پروژه در زبان C نوشته شد و بخش کلیدی پردازش وضعیت سنسور در زبان Assembly ARM پیاده‌سازی گردید. در بخشC ، عملیاتی مانند مقداردهی اولیه پورت‌ها، کنترلLED ها وBuzzer ، مدیریت حلقه اصلی برنامه و استفاده از SysTick برای زمان‌بندی انجام شد. در بخش Assembly نیز روتین‌هایی برای خواندن وضعیت ورودی، بررسی مقدار سنسور، ایجاد تأخیر نرم‌افزاری و ثبت نتیجه تشخیص حرکت نوشته شد.

در حالت عادی، سیستم هیچ حرکتی را تشخیص نمی‌دهد و LED سبز روشن است. این وضعیت نشان‌دهنده آماده بودن سیستم و نبود هشدار است. زمانی که سنسور PIR حرکت را تشخیص دهد، مقدار ورودی تغییر می‌کند و روتین Assemblyوضعیت آن را بررسی می‌کند. در صورت تأیید حرکت، مقدار متغیر motion_detectedتغییر کرده و برنامه C حالت هشدار را فعال می‌کند. در این حالت، LEDقرمز و Buzzer فعال شده و LED سبز خاموش می‌شود. پس از پایان زمان هشدار، سیستم دوباره به حالت عادی بازمی‌گردد.

در طول پروژه، مفاهیم مهمی مانندGPIO ، رجیسترهای سخت‌افزاری، معماریLoad/Store ، دستورات مقایسه و پرش شرطی، Stack، فراخوانی زیرروال، SysTick، حافظه Flash و SRAM و ارتباط بین C و Assembly مورد استفاده قرار گرفتند. همچنین پروژه در محیط Keil uVision پیاده‌سازی و Build شد و نتیجه نشان داد که فایل‌های C و Assembly به درستی در کنار هم کار می‌کنند.

در مجموع، پروژه توانست هدف آموزشی خود را برآورده کند. این سیستم نمونه‌ای ساده اما کاربردی از یک Embedded Systemاست که در آن ورودی محیط دریافت، داده پردازش و خروجی مناسب تولید می‌شود. همچنین می‌توان این پروژه را در آینده با اضافه کردن امکاناتی مانندLCD ، وقفه خارجی برای سنسورPIR ، ثبت زمان هشدار یا ارسال پیام توسعه داد.

8-2 . منابع

جزوه و مطالب ارائه‌شده در درس ریزپردازنده و زبان اسمبلی.

صورت پروژه پایانی عملی درس ریزپردازنده و زبان اسمبلی.

مستندات معماری ARM Cortex-M3 و مجموعه دستورهای Thumb / Thumb-2.

دیتاشیت و مستندات فنی میکروکنترلر STM32F103C8T6.

راهنمای نرم‌افزار Keil uVision برای ایجاد، Buildو Debug پروژه‌های ARM.

منابع آموزشی مربوط به سنسور PIR و کاربرد آن در سیستم‌های تشخیص حرکت.

فایل‌های پیاده‌سازی پروژه شامل main.cو motion_asm.s.